TP Final: Sistema de Turnos en C++

Desarrollar un programa en C++ que gestione turnos mediante dos entidades que se relacionan por la asignación de una fecha (ejemplo: doctores y pacientes en un consultorio; otros: profesores y alumnos, peluqueros y clientes, etc.). El programa se presenta y se defiende en la mesa de evaluación final.

Requisitos obligatorios

1. **POO:** dos entidades principales más una clase para el turno (la relación), con encapsulamiento, constructores y destructor, al menos una sobrecarga de operador y al menos una función `friend` justificada.
2. **Persistencia en archivos:** los datos de ambas entidades y de los turnos se guardan en archivos y se recuperan al iniciar el programa.
3. **Menú con ABML/CRUD completo** para las dos entidades y para los turnos (alta, baja, modificación, listado y consulta/búsqueda). Debe incluir baja lógica en al menos una entidad.
4. **Asignación de turnos con reglas de negocio:** no permitir duplicar un turno para la misma entidad en la misma fecha/hora, verificar que las entidades existan, permitir cancelar y reprogramar.
5. **Templates:** al menos una clase o función template reutilizada en el proyecto (ejemplo: manejador genérico de archivos o contenedor propio).
6. **Excepciones:** manejo de errores con `try/catch` y al menos una clase de excepción propia (ejemplo: archivo inexistente, ID duplicado, dato inválido, turno ocupado).
7. **Estructura de datos:** uso justificado de al menos una estructura vista en clase (lista, pila, cola o árbol), implementada por el alumno.
8. **Validación de entradas:** el programa no debe romperse ante datos inválidos (letras en lugar de números, fechas imposibles, campos vacíos).
9. **Organización y comentarios:** código dividido en archivos `.h/.cpp`, con comentarios que expliquen las decisiones principales.

Opcionales (suman a la nota)

- Cada entidad como librería importada al programa principal.
- Interfaz de consola cuidada (colores, tablas alineadas, limpieza de pantalla).
- Recursión aplicada de forma justificada (por ejemplo recorrido de un árbol).
- Reportes (turnos por fecha, por entidad, estadísticas).
- Persistencia de la estructura de datos.

Entrega y defensa

- Código fuente, archivos de datos de ejemplo y un README de una página con el diagrama de clases y la descripción de las decisiones de diseño.
- Defensa oral individual: explicación del código y una modificación pequeña en vivo.